6

Chapter 6 – Starting your own open source project

In this chapter you will apply all the knowledge you have acquired thus far to experience how to start your own open source project. So think of something big!

In this chapter you will learn about:

- How to set up the basic infrastructure of your project.
- How to gather people for your community.
- How to promote your software and get more contributors.
- How to handle huge amounts of change requests from users.

6.1 Starting your own open source project: The big picture

Before starting an open source software development project, you must perform a self assessment test using the flow chart depicted in *Figure 6.1*.

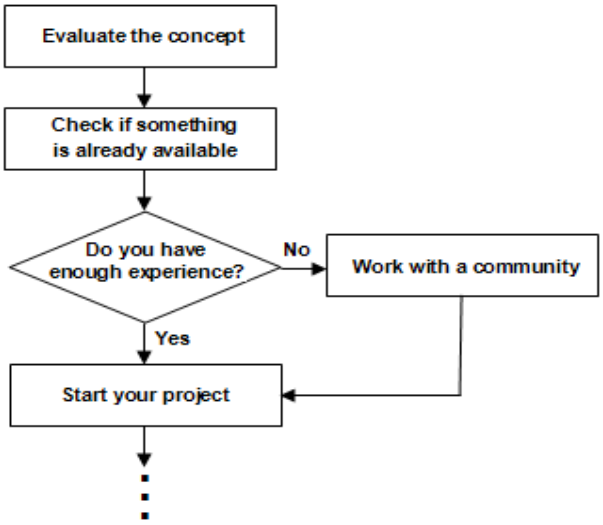


Figure 6.1 – Prerequisites for starting a new OSS project

As illustrated in *Figure 6.1*, the mandatory criteria for starting an OSS project is asking yourself two main questions:

1. What is the fundamental idea or requirement of your software? Do you feel many people would have same kind of requirements?
1. Is there any software already available in the market for the same purpose? If yes, does your product stand out for any reason? Why would people use it?

If you can convince yourself with the answers to the above questions, your project can be kicked off.

But, hold on. Do you have any prior experience contributing to an OSS project? Have you ever managed such a project? If your answer for either or both of these questions is NO, we suggest you be a part of at least one OSS project and understand their community culture. Get in touch with the management team and learn their experiences of dealing with various aspects of the project.

All sorted out? What are you waiting for then? Let's get started with setting up the platform for the project.

6.2 Providing the ecosystem for your open source project

An ***ecosystem*** is a perfect environment, where interactive species work together as a single unit. In an OSS project, this environment undoubtedly refers to the OSS community.

As you may have guessed, the intention of this section is to find out how to build a community. This may sound as something simple, but gathering contributors together, and getting the work done, is easier said than done. To achieve this goal, you need to have management skills and expertise. This may be a good approach:

1. Establish a goal for your project.
2. Create a plan, roadmap or strategy to accomplish your goal.
3. Find community members.
4. Understand the basic needs of your project and community.
5. Arrange for necessary funds.

The only way to get members for your community is to raise awareness. Is there any person in your social network, who you feel can help in developing the software? Yes? Go, and get him convinced to contribute to your project. Other activities that will help are:

- Posting your project details in relevant forums.
- Building an official Web site. This site should convey all the details of your project. It should be the only portal from where users can access the code repository, issue tracking database, see the build status, browse the mailing lists, and subscribe to one or more of them. A Web site will allow you to reach contributors from all over the world.

- Getting the members to promote the project. Once you get a few members for your community, encourage all of them to promote your community. This makes the awareness effort multiplied by the number of members.

The first and most important thing that an OSS product and project needs is an appropriate license. When choosing the license, remember that users always prefer those ones which are well known in the industry; for example, BSD, GPL, etc. An unfamiliar license, even if it is very short, simple, and straight forward may discourage some users from choosing your software and contribute in the project. So, it is always recommended to select an unchanged version of a well known, popular open source license unless your product has special characteristics that would require specific terms and conditions. Refer to *Chapter 3 - Licensing*, to get more detailed guidance on the choice of a license.

Setting up channels of communication is next in the list. As discussed before, the easiest way to mass communicate all the details of your project is through a Web site. You also need to create separate mailing lists, IRC chat rooms, and blogs where people can discuss about specific matters of the project or product. Your active participation as well as the participation from other key community members will often encourage the newbies to get more involved into the project, because it shows that the project management and the main developers have the interest to understand your feedback, and fix problems proactively. Refer to *Chapter 5 - Participating in open source development*, for more information about this topic.

Other requirements of the project include various software development tools for version control, issue tracking, automating build process, designing, coding and testing. We have discussed about these tools in *Chapter 4 - Community driven development*.

6.3 Accepting contributions

Chapter 5 gave you an idea on how you can contribute to an open source project. This section tells you, how to drive the car on your own. Growth of an OSS project strongly relies on code or some other contributions from the outer world. And as you have already learned, the only way to gather such contributions in large scale is to make your software popular among the users, so that they feel enthusiastic about participating effectively in your project. It is not very easy to make a product widely visible in the present competitive market. However, if the key concept is innovative and if your software is distinctly ahead of other existing products, it is definitely not impossible!

OSS development projects typically start with small communities. Since commitment to the project is completely voluntary, some developers may not always devote much time on your project when they are busy with other important jobs. In such situations, it becomes really challenging to take the work forward with limited amount of workforce. On the other hand, once your community becomes famous and well known, you can naturally expect to get flooded with bug reports, patches, functionality improvement request etc. on a daily basis. But keep your head on your shoulder. Never try to handle all these requests yourself. Rather, delegate it properly. Redirect the issues and other reports or requests to the relevant development team, and ask any of the people within that group to address it.

When someone contributes code as part of a code change or new feature request, a developer should first spend some time understanding the requirement. If things are not clear, he can request the contributor to provide more details. Once the idea is clear to the developer, he needs to check the status of the present release cycle to ensure if it is the right time to accept the request. If it is, the developer should walk through the code, before allowing the user to check it in. If it's not a good time, the request should be scheduled for the next release.

Dealing with code reviews is painful when a developer has to do it all by himself. Make sure to delegate and involve more people into the review work. You can also let the experienced contributors whom you have worked with earlier and are entrusted by their quality of work, commit their work directly. Besides balancing the workload throughout the entire community, it also helps to clear the bottlenecks by reducing the number of pending user requests.

Another suggestion is to share with the community the criteria to accept code contributions. This way the community will know ahead of time what is required and code accordingly.

6.4 Exercises

1. Become a member of an existing OSS community and use your practical knowledge and experience while driving your own project.
2. Sourceforge provides an integrated platform for developing open source software. Explore http://sourceforge.net/ before starting your OSS project. You may like to make use of it.

6.5 Summary

In this chapter, you learned how to start an open source software development project of your own. You understood how to build up the basic infrastructure of your project, how to form a community, and how to satisfy its various needs.

Later, the chapter discussed about handling multiple aspects of accepting user contributions, and how to get more and more people involved in the project with effective contributions. You also learned the way to deal with many user change requests.

6.6 Review questions

1. Which two fundamental points a founder of an OSS project must evaluate before starting his project?
2. What are the basic steps to start your OSS project?
3. How can you grow the awareness of your project?
4. Why is it recommended to go for an unchanged version of an already existing OSS license?
5. How can you overcome the stress of the code review effort?

6. Which of the following is not a valid method for getting members for an OSS community?

   A. Face to face interaction.

   B. Posting project details in relevant forums.

   C. Creating an official website.

   D. Consulting a legal advisor.

   E. None of the above

7. Which of the following is not a mandatory requirement to start an OSS project?

   A. A license.

   B. A software development company's guidance.

   C. Communication channels.

   D. Software development tools.

   E. All of the above

8. What is the next step after receiving a feature enhancement request from a user?

   A. Requirement analysis.

   B. Checking the status of the release cycle.

   C. Asking the user to deliver the necessary code.

   D. Postponing the improvement until next release.

   E. None of the above

9. Which of the following methods should you follow to handle overflowing user change requests?

   A. Let it be a responsibility of one particular member of the community.

   B. Delegate authority and tasks.

   C. Don't accept new change requests, when many of such requests are pending.

   D. None of the above.

   E. All of the above

10. What is the best possible way to minimize the effort of screening user contributed code?

    A. Allow any user to commit their changes directly to the project branch without reviewing them.

    B. Ask every user to perform all kinds of testing of their own.

    C. Sharing the code selection criteria with the users.

D. Accept code only from known external users.

E. None of the above